

ACTIVIDAD LABORATORIO NO.1
LABORATORIO: IMPLEMENTACIÓN DE UNA LISTA DOBLEMENTE
ENLAZADA

PRESENTADO POR:
ALEJANDRO DE MENDOZA

PRESENTADO AL PROFESOR:
ING EDWIN EDUARDO MILLAN ROJAS
PROFESOR

FUNDACIÓN UNIVERSITARIA INTERNACIONAL DE LA RIOJA
FACULTAD DE INGENIERÍA – INGENIERIA INFORMÁTICA
BOGOTÁ D.C.
9 DE MARZO
2025

Asignatura	Datos del alumno	Fecha
Estructura De Datos	Apellidos: De Mendoza	09/03/2024
	Nombre: Alejandro	

TABLA DE CONTENIDO

<u>RESUMEN.....</u>	<u>3</u>
<u>INTRODUCCIÓN</u>	<u>4</u>
<u>DESARROLLO DEL LABORATORIO.....</u>	<u>6</u>
CREACIÓN DEL PROYECTO EN NETBEANS	7
CREACIÓN DE PAQUETES DENTRO DEL PROYECTO EN NETBEANS	7
CREACIÓN DE LA CLASE NODO DE NOMBRE VEHÍCULO	8
CREACIÓN DE LA CLASE DE NOMBRE INVENTARIOVEHICULOS.....	12
CREACIÓN CLASE MAIN DE NOMBRE CONCESIONARIO VEHÍCULOS.....	22
RESULTADOS OBTENIDOS EN LA CONSOLA DE NETBEANS	34
<u>CONCLUSIÓN</u>	<u>35</u>
<u>BIBLIOGRAFÍA</u>	<u>38</u>

Asignatura	Datos del alumno	Fecha
Estructura De Datos	Apellidos: De Mendoza	09/03/2024
	Nombre: Alejandro	

RESUMEN

Para el desarrollo de este laboratorio del aula de Estructura de Datos me baso en la implementación de listas doblemente enlazadas en el lenguaje de programación Java utilizando la plataforma NetBeans en la aplicación a un caso real como es en la gestión del inventario de vehículos en un concesionario de alta gama. A lo largo del desarrollo, se construyó la clase `InventarioVehiculos`, que me permite administrar un inventario de automóviles mediante operaciones como agregar, eliminar, actualizar, buscar y fusionar vehículos de las dos diferentes sucursales.

Ahora, el código principal en `ConcesionarioVehiculos` lo que permite es la simulación y la administración de inventarios en distintas sucursales (en este caso desarrollé dos sucursales como ejemplo), probando las funcionalidades del sistema. Adicionalmente se desarrollaron métodos eficientes para manipular la lista de vehículos sin afectar su estructura y rendimiento, con base de igual manera en lo requerido por la actividad.

Finalizo este resumen e inicio del proceso documental del desarrollo indicando que el uso de listas doblemente enlazadas permite recorrer la lista en ambas direcciones y realizar inserciones y eliminaciones de manera óptima, facilitando la gestión dinámica del inventario lo que demuestra una aplicación práctica de estructuras de datos avanzadas en este caso aplicadas en la administración de concesionarios de automóviles.

Asignatura	Datos del alumno	Fecha
Estructura De Datos	Apellidos: De Mendoza	09/03/2024
	Nombre: Alejandro	

INTRODUCCIÓN

Como introducción y para la realización de este laboratorio se estudiarán los temas sobre listas doblemente enlazadas en Java diseñada específicamente para almacenar elementos de tipo String. Esta estructura de datos permitirá manipular cadenas de caracteres de manera eficiente, ofreciendo funcionalidades esenciales para su gestión y manipulación. Se aclara que este documento se ejecuta bajo la respectiva normatividad APA (Arial 12, Tamaño Carta, Interlineado 2.0, Márgenes 2.54, todas las imágenes han sido obtenidas de la plataforma Netbeans en su entorno de desarrollo por lo que no se considera necesario especificar en más detalle por extensión del trabajo).

Ahora, el objetivo principal de este primer laboratorio de estructura de datos es desarrollar una implementación personalizada de una lista doblemente enlazada sin utilizar código preexistente de la red, asegurando un entendimiento profundo de su funcionamiento. La lista incluirá operaciones clave como inserción, eliminación, búsqueda, conteo y concatenación de listas, lo que permitirá demostrar su versatilidad en aplicaciones prácticas.

Este documento técnico soporte junto con la implementación desarrollada en la plataforma NetBeans explicará las decisiones de diseño adoptadas, la arquitectura

Asignatura	Datos del alumno	Fecha
Estructura De Datos	Apellidos: De Mendoza	09/03/2024
	Nombre: Alejandro	

de la solución y los pasos necesarios en la implementación y prueba del desarrollo, que son los siguientes:

- Se debe incluir una clase que muestre el funcionamiento de la lista implementada sobre un problema real, utilizando todos los métodos implementados y mostrando por consola que el funcionamiento es correcto.
- Desarrollar una lista tipeada para almacenar elementos de tipo String, es decir, solo se almacenarán cadenas de caracteres.
- La funcionalidad que se debe incluir es:
 - Contar el número de elementos que tiene la lista.
 - Mostrar el elemento que hay en una posición concreta de la lista.
 - Comprobar si un elemento está en la lista.
 - Imprimir los elementos que contiene la lista.
 - Insertar un elemento nuevo.
 - Sacar un elemento concreto de la lista.
 - Sacar el elemento que ocupa una posición en la lista.
 - Concatenar dos listas.
 - Reemplazar un elemento de la lista.

Es por esto por lo que como primera medida y base para este trabajo sobre un problema real se ha decidido de mi parte generar un enfoque en el inventario de un concesionario de carros de alta gama con el fin que se puedan generar las manipulaciones idóneas en el desarrollo de la lista doblemente enlazada al ingresar,

Asignatura	Datos del alumno	Fecha
Estructura De Datos	Apellidos: De Mendoza	09/03/2024
	Nombre: Alejandro	

sacar, buscar, reemplazar, mostrar el inventario y concatenar inventarios de acuerdo con lo propuesto en este laboratorio.

Considero importante indicar en esta introducción que para este desarrollo me voy a basar en la plataforma **NetBeans** que es un entorno de desarrollo integrado (IDE) que me permite programar en varios lenguajes, principalmente Java como es el caso, aunque también es compatible con C, C++, PHP, HTML, JavaScript, y más. Es un software de código abierto, desarrollado originalmente por Sun Microsystems y ahora mantenido por la Apache Software Foundation solo como cultura general.

DESARROLLO DEL LABORATORIO

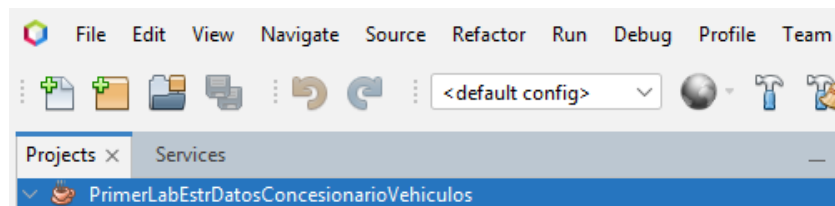
En este caso mi enfoque y como se dio un breve abrebocas en la introducción como escenario real se ha tomado un concesionario de carros de alta gama porque nos da un claro ejemplo de utilización real y práctico además de relevante para la implementación de listas dobles enlazadas en este caso en Java. Es de claridad que un concesionario de vehículos maneja diferentes tipos de vehículos y estos pueden ser agregados, eliminados, buscados, actualizados y organizados, lo que de mi parte se considera encaja correctamente con la temática que debemos desarrollar en este caso de una lista doblemente enlazada.

Asignatura	Datos del alumno	Fecha
Estructura De Datos	Apellidos: De Mendoza	09/03/2024
	Nombre: Alejandro	

Sin más preámbulos y con el fin de mi parte de dar inicio al desarrollo de este laboratorio espero sea de gran provecho para el profesor la lectura y análisis de la resolución de este desarrollado ejecutado de mi parte.

Creación Del Proyecto En NETBEANS

Entonces para este desarrollo lo primero de mi parte fue abrir la plataforma de NetBeans y crear el proyecto de Java que en este caso recibe el nombre de “PrimerLabEstrDatosConcesionarioVehiculos”. A continuación, muestro la imagen respectiva de la creación del proyecto en el lenguaje de Java:



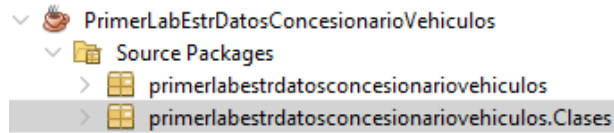
Creación De Paquetes Dentro Del Proyecto En NETBEANS

Luego de crear el proyecto se procede a crear dos paquetes que van a contener las respectivas clases y que van a estar incluidos dentro de los Source Packages (Src) del proyecto, los nombres de los paquetes creados son:

- primerlabestrdatosconcesionariovehiculos
- primerlabestrdatosconcesionariovehiculos.Clases

A continuación, la imagen respectiva:

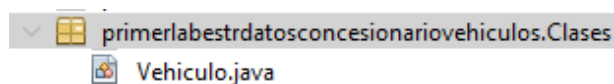
Asignatura	Datos del alumno	Fecha
Estructura De Datos	Apellidos: De Mendoza	09/03/2024
	Nombre: Alejandro	



Creación De La Clase Nodo De Nombre Vehículo

Luego de crear los paquetes entonces procedemos a crear una clase de nodo dentro del paquete “**primerlabestrdatosconcesionariovehiculos.Clases**” que se va a llamar en este caso “**Vehículo**” que representa un nodo dentro de una lista doblemente enlazada en Java. Esta clase almacena un modelo de vehículo y referencias a los nodos anterior y siguiente, lo que permite la navegación en ambas direcciones dentro de la lista, lo que concuerda con lo visto en la clase virtual del laboratorio del aula.

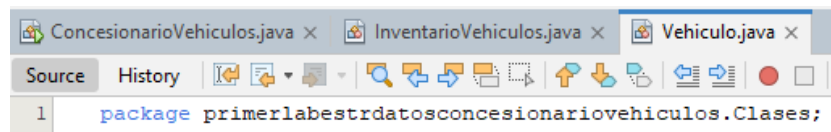
A continuación, la imagen respectiva:



Ahora entonces se procede con el desarrollo de su código e implementación por lo que a continuación, voy a brindar la explicación completa del código utilizado en el desarrollo de esta clase:

Asignatura	Datos del alumno	Fecha
Estructura De Datos	Apellidos: De Mendoza	09/03/2024
	Nombre: Alejandro	

1. Lo primero es definir el paquete en el que se encuentra la clase y en este caso, `primerlabestrdatosconcesionariovehiculos.Clases` es el paquete e indica que el código forma parte de un proyecto sobre estructuras de datos para un concesionario de vehículos. A continuación, la imagen respectiva:

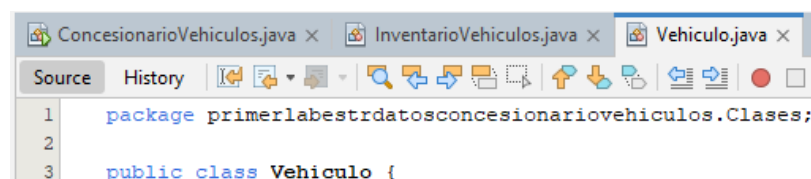


```

ConcesionarioVehiculos.java x InventarioVehiculos.java x Vehiculo.java x
Source History
1 package primerlabestrdatosconcesionariovehiculos.Clases;

```

2. Ahora se procede a declarar la clase Vehiculo, que es la que utilizo como el nodo en la lista doblemente enlazada, como explique en su anterioridad. Es importante indicar que en este caso la declare como pública con el fin que pueda ser accedida desde otras clases dentro del proyecto. A continuación, la imagen respectiva:



```

ConcesionarioVehiculos.java x InventarioVehiculos.java x Vehiculo.java x
Source History
1 package primerlabestrdatosconcesionariovehiculos.Clases;
2
3 public class Vehiculo {

```

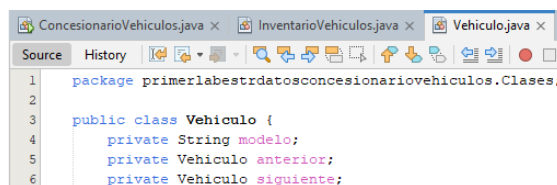
3. Ahora procedí a desarrollar los atributos de esta clase Vehiculo los cuales consisten en:

- modelo: Este atributo almacena el nombre o modelo del vehículo (por ejemplo, "Toyota Corolla").

Asignatura	Datos del alumno	Fecha
Estructura De Datos	Apellidos: De Mendoza	09/03/2024
	Nombre: Alejandro	

- anterior: Este atributo referencia al vehículo anterior en la lista (nodo previo).
- siguiente: Finalmente este atributo referencia al vehículo siguiente en la lista (nodo siguiente).

A continuación, la imagen respectiva:

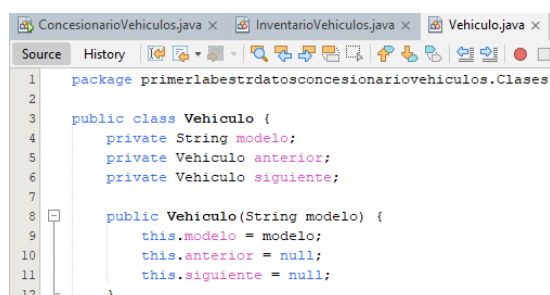


```

1 package primerlabestrdatosconcesionariovehiculos.Clases;
2
3 public class Vehiculo {
4     private String modelo;
5     private Vehiculo anterior;
6     private Vehiculo siguiente;

```

- Después de esto procedo con el desarrollo del constructor el cual inicializa un nuevo nodo (vehículo) con un modelo específico concordando con lo indicado en la clase, donde el profesor muestra cómo crear un constructor de una manera fácil e idónea en tiempo. Es importante indicar que en adición y de acuerdo con lo indicado establece anterior y siguiente en null, porque cuando se crea, el nodo aún no está conectado a otros. A continuación, la respectiva imagen:



```

1 package primerlabestrdatosconcesionariovehiculos.Clases;
2
3 public class Vehiculo {
4     private String modelo;
5     private Vehiculo anterior;
6     private Vehiculo siguiente;
7
8     public Vehiculo(String modelo) {
9         this.modelo = modelo;
10        this.anterior = null;
11        this.siguiente = null;
12    }

```

Asignatura	Datos del alumno	Fecha
Estructura De Datos	Apellidos: De Mendoza	09/03/2024
	Nombre: Alejandro	

5. Luego procedo con el desarrollo de los métodos Getter y Setter de acuerdo con lo especificado en el aula, lo cual como indico gracias a la explicación del profesor donde con clic derecho + insertar código + constructor + seleccionando cabeza y cola respectivamente procedemos con su creación, ahora los métodos Get y Set van a desarrollar las siguientes acciones:

➤ El método “Get” desarrolla las siguientes acciones:

- Obtiene el modelo del vehículo
- Obtiene el nodo anterior. Retorna el vehículo anterior en la lista.
- Obtiene el nodo siguiente. Retorna el vehículo siguiente en la lista.

➤ El método “Set” desarrolla las siguientes acciones:

- Procede con el cambio del modelo
- Establece el nodo del vehículo anterior, conectando este nodo con otro.
- Establece el nodo del vehículo siguiente, conectando este nodo con otro.

A continuación, la imagen respectiva del código:

Asignatura	Datos del alumno	Fecha
Estructura De Datos	Apellidos: De Mendoza	09/03/2024
	Nombre: Alejandro	

```

ConcesionarioVehiculos.java x InventarioVehiculos.java x Vehiculo.java x
Source History
1 package primerlabestrdatosconcesionariovehiculos.Clases;
2
3 public class Vehiculo {
4     private String modelo;
5     private Vehiculo anterior;
6     private Vehiculo siguiente;
7
8     public Vehiculo(String modelo) {
9         this.modelo = modelo;
10        this.anterior = null;
11        this.siguiente = null;
12    }
13
14    public String getModelo() {
15        return modelo;
16    }
17
18    public void setModelo(String modelo) {
19        this.modelo = modelo;
20    }
21
22    public Vehiculo getAnterior() {
23        return anterior;
24    }
25
26    public void setAnterior(Vehiculo anterior) {
27        this.anterior = anterior;
28    }
29
30    public Vehiculo getSiguiente() {
31        return siguiente;
32    }
33
34    public void setSiguiente(Vehiculo siguiente) {
35        this.siguiente = siguiente;
36    }
37 }

```

6. Y con esto finalizo la creación de la clase nodo de nombre Vehículo.
7. Ahora procedo con el desarrollo de la clase de nombre “InventarioVehiculos”, la cual va a ser la clase que me va a gestionar el inventario de los vehículos del concesionario utilizando una lista doblemente enlazada. Esta estructura de datos o clase es que me permite agregar, eliminar, actualizar y consultar vehículos de manera eficiente y se aclara se debe construir junto con el main, pero las voy a explicar por separado por mayor entendimiento y simplicidad en el desarrollo de este laboratorio.

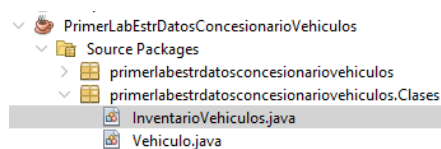
Creación De La Clase De Nombre InventarioVehiculos

Entonces, como ya he procedido a crear la clase Nodo de nombre **Vehiculo** ahora lo siguiente es crear la siguiente clase de igual manera dentro del paquete “**primerlabestrdatosconcesionariovehiculos.Clases**” que en este caso la voy a

Asignatura	Datos del alumno	Fecha
Estructura De Datos	Apellidos: De Mendoza	09/03/2024
	Nombre: Alejandro	

denotar con el nombre de **InventarioVehiculos** que es la clase que en este caso me representa una lista doblemente enlazada de vehículos, y como indique en el anterior punto me permite desarrollar operaciones como agregar, eliminar, buscar, actualizar y fusionar inventarios, con base en los procesos requeridos a desarrollar en este laboratorio frente a este desarrollo.

A continuación, la imagen respectiva:

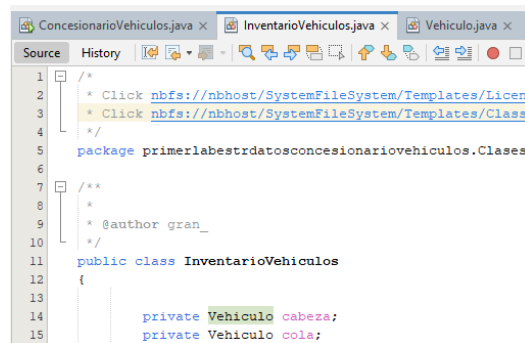


A continuación, procedo entonces a brindar la explicación completa del código utilizado en el desarrollo de esta clase:

1. Lo primero de mi parte fue desarrollar los atributos de la clase y de acuerdo con lo indicado por el profesor lo mejor siempre es dejar los atributos en privado, aunque impliquen un poco más de desarrollo y para esto se desarrollaron los atributos Vehiculo cabeza y Vehículo cola donde:
 - cabeza: Es el atributo que apunta al primer vehículo en la lista.
 - cola: Este atributo apunta al último vehículo en la lista.
 - Es importante indicar que, si la lista está vacía, cabeza y cola son null.

Asignatura	Datos del alumno	Fecha
Estructura De Datos	Apellidos: De Mendoza	09/03/2024
	Nombre: Alejandro	

A continuación, la imagen respectiva:

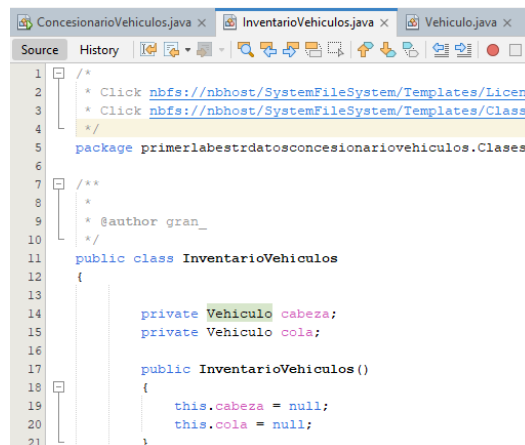


```

1  /*
2  * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses
3  * Click nbfs://nbhost/SystemFileSystem/Templates/Classes
4  */
5  package primerlabestrdatosconcesionariovehiculos.Clases;
6
7  /**
8   *
9   * @author gran_
10  */
11  public class InventarioVehiculos
12  {
13
14      private Vehiculo cabeza;
15      private Vehiculo cola;

```

- Ahora procedo a crear el constructor el cual se inicializa con la lista como vacía, el proceso de creación lo indique en su anterioridad en el punto 5 de la clase anteriormente creada. A continuación, la imagen respectiva:



```

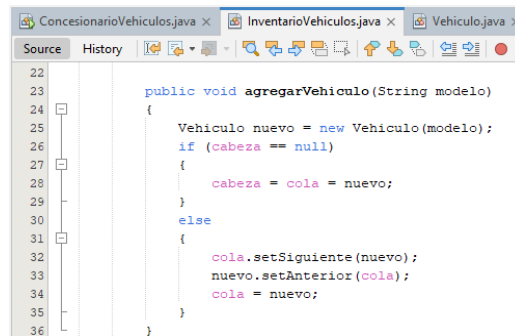
1  /*
2  * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses
3  * Click nbfs://nbhost/SystemFileSystem/Templates/Classes
4  */
5  package primerlabestrdatosconcesionariovehiculos.Clases;
6
7  /**
8   *
9   * @author gran_
10  */
11  public class InventarioVehiculos
12  {
13
14      private Vehiculo cabeza;
15      private Vehiculo cola;
16
17      public InventarioVehiculos ()
18      {
19          this.cabeza = null;
20          this.col = null;
21      }

```

- Ahora se procedió a generar el método `agregarVehiculo(String modelo)`, el cual crea un nuevo nodo `Vehiculo` con el modelo dado. Ahora si la lista esta vacía, entonces `cabeza` y `cola` apuntan hacia el nuevo vehículo, por otra parte, si la lista ya tiene vehículos entonces lo que hace el método es que enlaza el nuevo

Asignatura	Datos del alumno	Fecha
Estructura De Datos	Apellidos: De Mendoza	09/03/2024
	Nombre: Alejandro	

nodo al final (cola) y se actualiza la referencia cola al nuevo nodo. A continuación, la imagen respectiva para dar un mayor entendimiento:

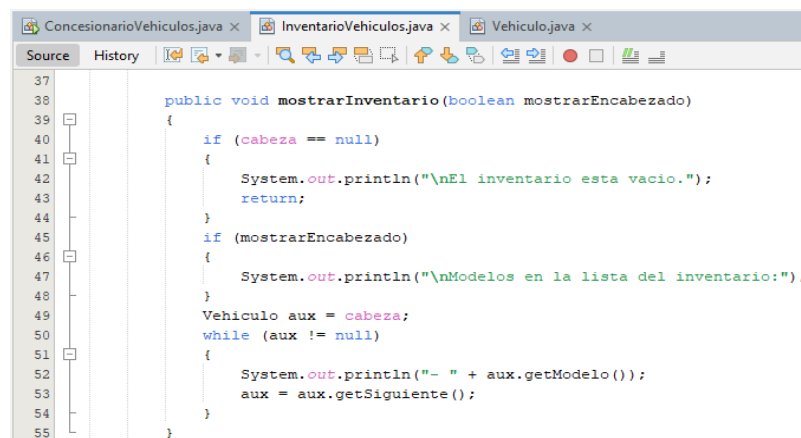


```

22
23
24 public void agregarVehiculo(String modelo)
25 {
26     Vehiculo nuevo = new Vehiculo(modelo);
27     if (cabeza == null)
28     {
29         cabeza = cola = nuevo;
30     }
31     else
32     {
33         cola.setSiguiente(nuevo);
34         nuevo.setAnterior(cola);
35         cola = nuevo;
36     }
37 }

```

4. Ahora se procedió de mi parte entonces a generar el método **mostrarInventario(boolean mostrarEncabezado)**, el cual indica que si la lista está vacía, muestra un mensaje “El inventario está vacío”, por otra parte si mostrarEncabezado es true, imprime el título “Modelos en la lista del inventario”. Y finalmente recorre la lista desde cabeza, imprimiendo los modelos. A continuación, la imagen respectiva:



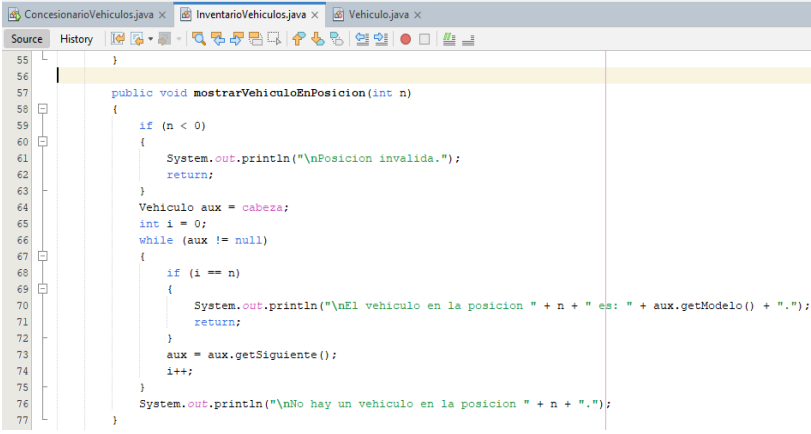
```

37
38
39 public void mostrarInventario(boolean mostrarEncabezado)
40 {
41     if (cabeza == null)
42     {
43         System.out.println("\nEl inventario esta vacio.");
44         return;
45     }
46     if (mostrarEncabezado)
47     {
48         System.out.println("\nModelos en la lista del inventario:");
49     }
50     Vehiculo aux = cabeza;
51     while (aux != null)
52     {
53         System.out.println("- " + aux.getModelo());
54         aux = aux.getSiguiente();
55     }
56 }

```

Asignatura	Datos del alumno	Fecha
Estructura De Datos	Apellidos: De Mendoza	09/03/2024
	Nombre: Alejandro	

5. Ahora y con base en la actividad se procedió a desarrollar el método **mostrarVehiculoEnPosicion(int n)**, el cual es un método que recorre la lista hasta la posición “n”, (la posición que se indique en el main) y si encuentra un vehículo en esa posición, lo muestra junto con su posición como se puede ver en el título. Ahora en dado caso que, si “n” es negativo o no hay suficientes vehículos, muestra un mensaje de error indicando que no hay un vehículo en esa posición. A continuación, la imagen respectiva:



```

55 }
56
57 public void mostrarVehiculoEnPosicion(int n)
58 {
59     if (n < 0)
60     {
61         System.out.println("\nPosicion invalida.");
62         return;
63     }
64     Vehiculo aux = cabeza;
65     int i = 0;
66     while (aux != null)
67     {
68         if (i == n)
69         {
70             System.out.println("\nEl vehiculo en la posicion " + n + " es: " + aux.getModelo() + ".");
71             return;
72         }
73         aux = aux.getSiguiente();
74         i++;
75     }
76     System.out.println("\nNo hay un vehiculo en la posicion " + n + ".");
77 }

```

6. Se procede entonces de mi parte a desarrollar el método **contarVehiculos()**, el cual es un método que simplemente cuenta cuantos vehículos hay en la lista y los imprime con el título “El número total de vehículos en la lista del el inventario”, a continuación la imagen respectiva:

Asignatura	Datos del alumno	Fecha
Estructura De Datos	Apellidos: De Mendoza	09/03/2024
	Nombre: Alejandro	

```

79 public void contarVehiculos ()
80 {
81     int count = 0;
82     Vehiculo aux = cabeza;
83     while (aux != null)
84     {
85         count++;
86         aux = aux.getSiguiente();
87     }
88     System.out.println("\nEl numero total de vehiculos en la lista de el inventario: " + count + ".");
89 }

```

7. Se procede ahora a desarrollar el método **verificarVehiculo(String modelo)**, el cual es un método que busca un vehículo por modelo, se aclara que en este código se ignoran mayúsculas/minúsculas y devuelve un valor booleano de true si lo encuentra emitiendo un título que indica “El vehículo (modelo) está en el inventario”, false si no y en dado caso se emite un título “El vehículo (modelo) no está en el inventario”. A continuación, la respectiva imagen:

```

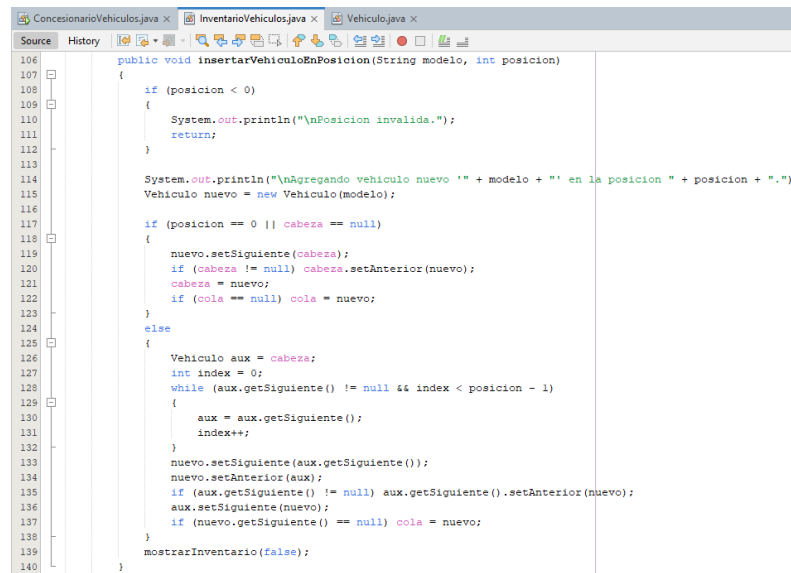
91 public boolean verificarVehiculo(String modelo) {
92     Vehiculo aux = cabeza;
93     while (aux != null)
94     {
95         if (aux.getModelo().equalsIgnoreCase(modelo))
96         {
97             System.out.println("\nEl vehiculo '" + modelo + "' esta en el inventario.");
98             return true;
99         }
100         aux = aux.getSiguiente();
101     }
102     System.out.println("\nEl vehiculo '" + modelo + "' no está en el inventario.");
103     return false;
104 }

```

8. Se procede con el desarrollo entonces del método **insertarVehiculoEnPosicion(String modelo, int posicion)**, y este método inserta un vehículo en una posición específica (la posición que se indique en el main del cual explicare su desarrollo más adelante) y cuando el vehículo nuevo es insertado se muestra un mensaje de “Agregando vehículo nuevo (modelo) en la posición (La que se indique)”. Ahora si la posición es 0, lo inserta al inicio

Asignatura	Datos del alumno	Fecha
Estructura De Datos	Apellidos: De Mendoza	09/03/2024
	Nombre: Alejandro	

y claramente si la posición es mayor, lo inserta en su lugar correspondiente. A continuación, la imagen respectiva del código:



```

106
107
108 {
109     if (posicion < 0)
110     {
111         System.out.println("\nPosicion invalida.");
112         return;
113     }
114
115     System.out.println("\nAgregando vehiculo nuevo '" + modelo + "' en la posicion " + posicion + ".");
116     Vehiculo nuevo = new Vehiculo(modelo);
117
118     if (posicion == 0 || cabeza == null)
119     {
120         nuevo.setSiguiente(cabeza);
121         if (cabeza != null) cabeza.setAnterior(nuevo);
122         cabeza = nuevo;
123         if (cola == null) cola = nuevo;
124     }
125     else
126     {
127         Vehiculo aux = cabeza;
128         int index = 0;
129         while (aux.getSiguiente() != null && index < posicion - 1)
130         {
131             aux = aux.getSiguiente();
132             index++;
133         }
134         nuevo.setSiguiente(aux.getSiguiente());
135         nuevo.setAnterior(aux);
136         if (aux.getSiguiente() != null) aux.getSiguiente().setAnterior(nuevo);
137         aux.setSiguiente(nuevo);
138         if (nuevo.getSiguiente() == null) cola = nuevo;
139     }
140     mostrarInventario(false);
141 }

```

- Ahora se procede a desarrollar el método de eliminación de un vehículo, este método lo denomine como **eliminarVehiculo(String modelo)**, el cual es un método que como su nombre lo indica busca un vehículo por modelo y lo elimina. Ahora si está en la cabeza, mueve la cabeza al siguiente y por el contrario parte si está en la cola, mueve la cola al anterior, pero si esta en el medio ajusta los punteros. A continuación, la imagen respectiva:

Asignatura	Datos del alumno	Fecha
Estructura De Datos	Apellidos: De Mendoza	09/03/2024
	Nombre: Alejandro	

```

142 public void eliminarVehiculo(String modelo)
143 {
144     if (cabeza == null)
145     {
146         System.out.println("\nEl inventario esta vacio.");
147         return;
148     }
149
150     Vehiculo aux = cabeza;
151     while (aux != null && !aux.getModelo().equalsIgnoreCase(modelo))
152     {
153         aux = aux.getSiguiente();
154     }
155
156     if (aux == null)
157     {
158         System.out.println("\nEl vehiculo '" + modelo + "' no esta en el inventario.");
159         return;
160     }
161
162     if (aux == cabeza)
163     {
164         cabeza = cabeza.getSiguiente();
165         if (cabeza != null) cabeza.setAnterior(null);
166     }
167     else if (aux == cola)
168     {
169         cola = cola.getAnterior();
170         if (cola != null) cola.setSiguiente(null);
171     }
172     else
173     {
174         aux.getAnterior().setSiguiente(aux.getSiguiente());
175         aux.getSiguiente().setAnterior(aux.getAnterior());
176     }
177
178     System.out.println("\nSe ha eliminado el vehiculo '" + modelo + "' como elemento concreto de la lista de vehiculos.");
179     mostrarInventario(false);
180 }

```

10. Ahora y con el fin de continuar con los puntos del laboratorio se procede a desarrollar el método de **eliminarVehiculoEnPosicion(int posicion)**, el cual es un método que lo que hace es primero verifica si la lista está vacía, luego verifica si la posición es válida, luego recorre la lista hasta la posición dada y si no encuentra un vehículo en esa posición, avisa que está fuera de rango. Ahora si el vehículo a eliminar es la cabeza, ajusta la referencia y por otra parte si es la cola, se ajusta la referencia. Ahora en dado caso que se encuentre en el medio ajusta los punteros de los nodos vecinos y finalmente imprime el nuevo inventario después de la eliminación. A continuación, la imagen respectiva del código:

Asignatura	Datos del alumno	Fecha
Estructura De Datos	Apellidos: De Mendoza	09/03/2024
	Nombre: Alejandro	

```

181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226

public void eliminarVehiculoEnPosicion(int posicion)
{
    if (cabeza == null)
    {
        System.out.println("\nEl inventario esta vacio.");
        return;
    }
    if (posicion < 0)
    {
        System.out.println("\nPosicion invalida.");
        return;
    }

    Vehiculo aux = cabeza;
    int index = 0;
    while (aux != null && index < posicion)
    {
        aux = aux.getSiguiente();
        index++;
    }

    if (aux == null)
    {
        System.out.println("\nLa posicion " + posicion + " esta fuera de rango.");
        return;
    }

    if (aux == cabeza)
    {
        cabeza = cabeza.getSiguiente();
        if (cabeza != null) cabeza.setAnterior(null);
    } else if (aux == cola)
    {
        cola = cola.getAnterior();
        if (cola != null) cola.setSiguiente(null);
    }
    else
    {
        aux.getAnterior().setSiguiente(aux.getSiguiente());
        aux.getSiguiente().setAnterior(aux.getAnterior());
    }

    System.out.println("\nSe elimino el vehiculo en la posicion " + posicion + ".");
    mostrarInventario(false);
}

```

11. Ahora de mi parte se creó el método como proceso de concatenación de listas y de nombre **fusionarInventarios(InventarioVehiculos otroInventario)**, el cual es un método que une las dos listas de los vehículos denotadas con los nombres de **inventarioSucursal1** y **inventarioSucursal2** (estas listas son desarrolladas en el main y las explicaré más adelante), ahora si la primera lista (inventarioSucursal1), está vacía entonces la reemplaza con la segunda lista, y en caso contrario concatena la segunda lista (inventarioSucursal2) al final. A continuación, la respectiva imagen del código:

Asignatura	Datos del alumno	Fecha
Estructura De Datos	Apellidos: De Mendoza	09/03/2024
	Nombre: Alejandro	

```

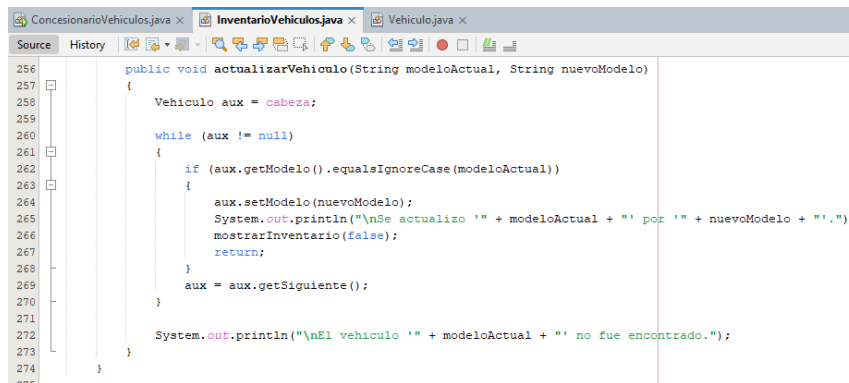
228 public void fusionarInventarios(InventarioVehiculos otroInventario)
229 {
230     if (otroInventario.cabeza == null) return;
231
232     if (this.cabeza == null)
233     {
234         this.cabeza = otroInventario.cabeza;
235         this.cola = otroInventario.cola;
236     }
237     else
238     {
239         this.cola.setSiguiente(otroInventario.cabeza);
240         otroInventario.cabeza.setAnterior(this.cola);
241         this.cola = otroInventario.cola;
242     }
243     System.out.println("\nInventarios fusionados o listas de inventarios concatenadas:");
244 }

```

12. Por último y para culminar el desarrollo de esta clase, se creó el método de **actualizarVehiculo(String modeloActual, String nuevoModelo)**, el cual es un método que crea un puntero aux y se inicializa con la cabeza de la lista y el objetivo del mismo es recorrer la lista buscando el vehículo a actualizar. Luego se procede a utilizar un while para recorrer los nodos de la lista mientras aux no sea null, ahora dentro del while, lo que hace el método es comparar el modelo del vehículo actual (aux.getModelo()) con el modeloActual que se quiere actualizar. Luego y algo que considero importante se implementó es que se dio de mi parte la implementación de equalsIgnoreCase() para evitar problemas con mayúsculas/minúsculas esto para evitar errores y hace el código más eficiente. Entonces, si el vehículo es encontrado, se cambia su modelo con aux.setModelo(nuevoModelo) y se imprime un mensaje de confirmación "Se actualizó (el modelo actual) por (el nuevo modelo)". Luego se llama a mostrarInventario(false) para mostrar la lista actualizada y finalmente se usa return para salir del método, ya que la actualización está completa. Es importante precisar que, si aux llega a null sin haber encontrado coincidencias,

Asignatura	Datos del alumno	Fecha
Estructura De Datos	Apellidos: De Mendoza	09/03/2024
	Nombre: Alejandro	

se imprime un mensaje indicando que el vehículo no fue encontrado. A continuación, la respectiva imagen del código:



```

256 public void actualizarVehiculo(String modeloActual, String nuevoModelo)
257 {
258     Vehiculo aux = cabeza;
259
260     while (aux != null)
261     {
262         if (aux.getModelo().equalsIgnoreCase(modeloActual))
263         {
264             aux.setModelo(nuevoModelo);
265             System.out.println("\nSe actualizo '" + modeloActual + "' por '" + nuevoModelo + "'.");
266             mostrarInventario(false);
267             return;
268         }
269         aux = aux.getSiguiente();
270     }
271
272     System.out.println("\nEl vehiculo '" + modeloActual + "' no fue encontrado.");
273 }
274

```

Creación Clase Main De Nombre Concesionario Vehículos

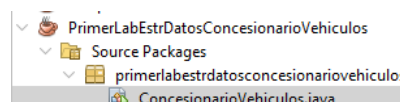
Ya que he procedido a dar a conocer el desarrollo de la creación de las clases del paquete de “primerlabestrdatosconcesionariovehiculos.Clases”, a continuación, voy a describir el desarrollo de la clase main que denote con el nombre de **ConcesionarioVehiculos**. Este programa lo que hace es simular la gestión de inventarios de vehículos en un concesionario. Utiliza una estructura de datos para almacenar y manipular listas de vehículos de dos distintas sucursales que en este caso denote con los nombres de **inventarioSucursal1** y **inventarioSucursal2** (*la creación de las listas de inventarios de estas dos sucursales las explico más adelante en desde el punto 5 de la creación de esta clase*). Entonces las funcionalidades que va a ejecutar este programa a través de esta clase son las siguientes:

- Agregar vehículos a un inventario.

Asignatura	Datos del alumno	Fecha
Estructura De Datos	Apellidos: De Mendoza	09/03/2024
	Nombre: Alejandro	

- Contar la cantidad de vehículos.
- Consultar un vehículo en una posición específica.
- Verificar si un vehículo está en la lista.
- Mostrar el inventario completo.
- Insertar y eliminar vehículos en posiciones específicas.
- Fusionar dos inventarios de diferentes sucursales.
- Actualizar el nombre de un vehículo en el inventario.

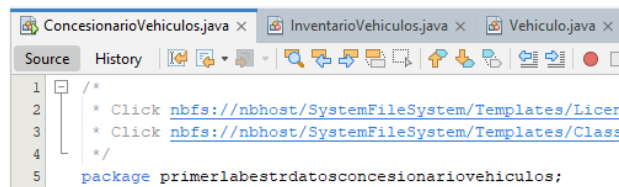
Es importante precisar de mi parte que considero que este código es útil para tener claridad del trabajo a realizar con listas enlazadas en Java y cómo organizar datos de manera eficiente. A continuación, la imagen respectiva de la creación de la clase:



A continuación, procedo entonces con la explicación del desarrollo del código de esta clase main:

1. Lo primero entonces es entrar a definir el paquete donde está ubicado el paquete dentro del código que en este caso es primerlabestrdatosconcesionariovehiculos. A continuación, la imagen respectiva:

Asignatura	Datos del alumno	Fecha
Estructura De Datos	Apellidos: De Mendoza	09/03/2024
	Nombre: Alejandro	

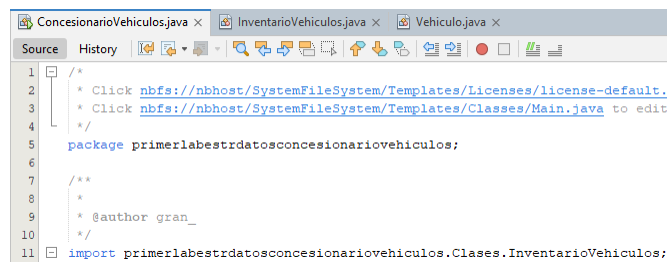


```

1  /*
2  * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to
3  * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit
4  */
5  package primerlabestrdatosconcesionariovehiculos;

```

2. Ahora procedo a importar la clase InventarioVehiculos, que es como sabemos y se explicó en los anteriores puntos la que gestiona la lista de inventario de vehículos en el concesionario. A continuación, la imagen respectiva:

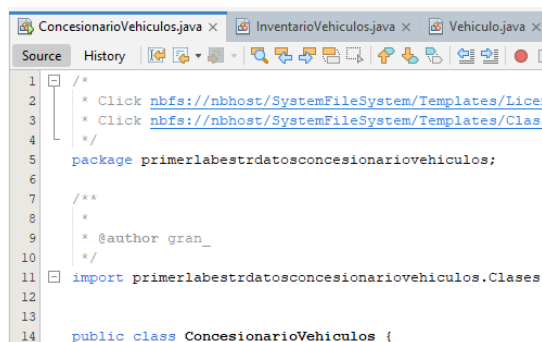


```

1  /*
2  * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to
3  * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit
4  */
5  package primerlabestrdatosconcesionariovehiculos;
6
7  /**
8   *
9   * @author gran_
10  */
11 import primerlabestrdatosconcesionariovehiculos.Clases.InventarioVehiculos;

```

3. Ahora procedí a declarar la clase principal del programa (main) que en este caso la denoté con el nombre de **ConcesionarioVehiculos** y que como es claro contiene el método main(), que es el punto de entrada del programa. A continuación, la imagen respectiva:



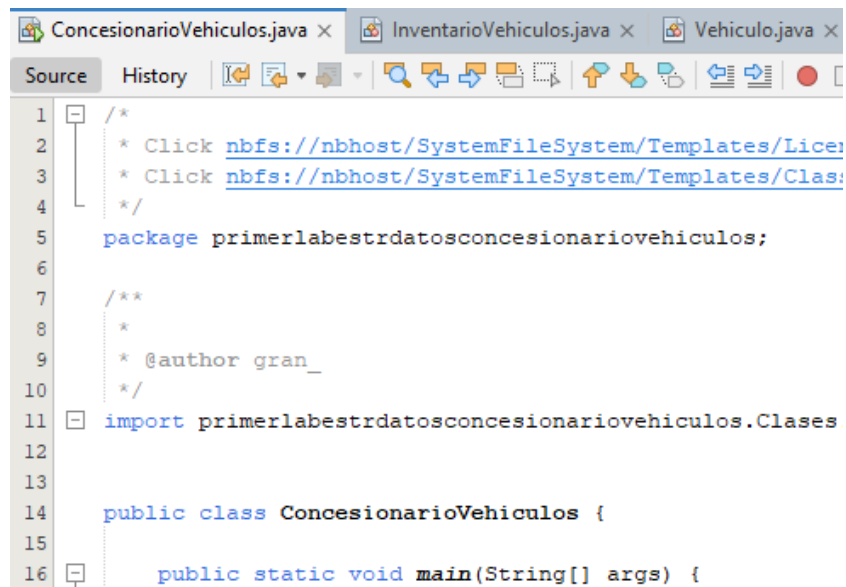
```

1  /*
2  * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to
3  * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit
4  */
5  package primerlabestrdatosconcesionariovehiculos;
6
7  /**
8   *
9   * @author gran_
10  */
11 import primerlabestrdatosconcesionariovehiculos.Clases.InventarioVehiculos;
12
13
14 public class ConcesionarioVehiculos {

```


Asignatura	Datos del alumno	Fecha
Estructura De Datos	Apellidos: De Mendoza	09/03/2024
	Nombre: Alejandro	

4. Ahora procedo a incluir en el código el método principal donde se ejecutan las acciones del programa que es “public static void main(String[] args)”. A continuación, la respectiva imagen:



```

1  /*
2   * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses
3   * Click nbfs://nbhost/SystemFileSystem/Templates/Classes
4   */
5  package primerlabestrdatosconcesionariovehiculos;
6
7  /**
8   *
9   * @author gran_
10  */
11 import primerlabestrdatosconcesionariovehiculos.Clases
12
13
14 public class ConcesionarioVehiculos {
15
16     public static void main(String[] args) {

```

5. Procedo entonces con la creación del primer inventario de vehículos y para esto se crea un objeto **inventarioSucursal1**, que representa la lista de vehículos de la primera sucursal, con los vehículos de nombre Bugatti, Ferrari, Porsche y Maserati. Y cada vehículo se almacena como un nodo en una lista enlazada. A continuación, la imagen respectiva:

Asignatura	Datos del alumno	Fecha
Estructura De Datos	Apellidos: De Mendoza	09/03/2024
	Nombre: Alejandro	

```

ConcesionarioVehiculos.java x InventarioVehiculos.java x Vehiculo.java x
Source History
1  /*
2   * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.t
3   * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Main.java to edit t
4   */
5   package primerlabestrdatosconcesionariovehiculos;
6
7   /**
8    *
9    * @author gran_
10   */
11  import primerlabestrdatosconcesionariovehiculos.Clases.InventarioVehiculos;
12
13
14  public class ConcesionarioVehiculos {
15
16      public static void main(String[] args) {
17          InventarioVehiculos inventarioSucursal1 = new InventarioVehiculos();
18          inventarioSucursal1.agregarVehiculo("Bugatti");
19          inventarioSucursal1.agregarVehiculo("Ferrari");
20          inventarioSucursal1.agregarVehiculo("Porsche");
21          inventarioSucursal1.agregarVehiculo("Maserati");
22

```

6. Luego procedo para ejecutar el punto de concatenación a crear el segundo inventario de vehículos y para esto se crea otro objeto **inventarioSucursal2**, representando una segunda sucursal y donde se agregan cuatro vehículos (Alfa Romeo, Mercedes, BMW, Mazda). A continuación, la respectiva imagen:

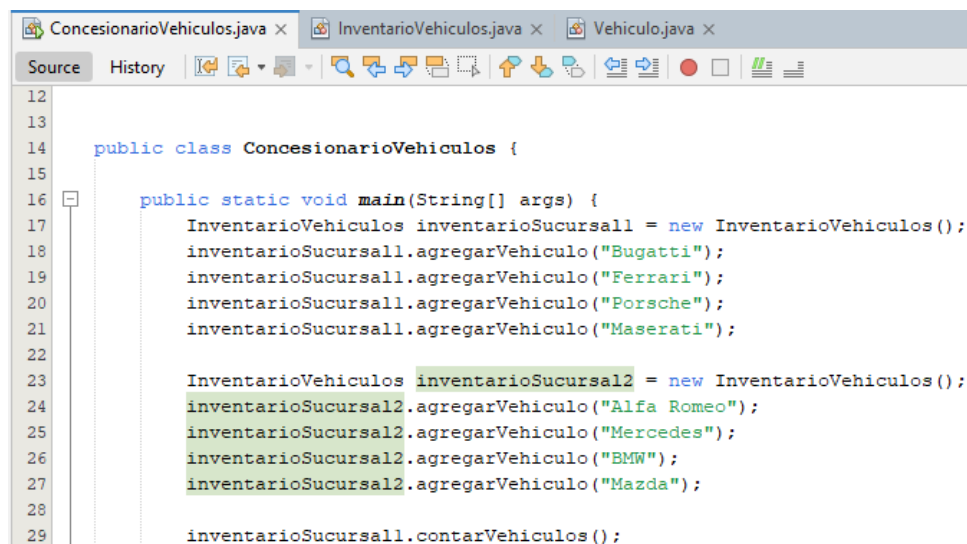
```

ConcesionarioVehiculos.java x InventarioVehiculos.java x Vehiculo.java x
Source History
1  /*
2   * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.t
3   * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Main.java to edit t
4   */
5   package primerlabestrdatosconcesionariovehiculos;
6
7   /**
8    *
9    * @author gran_
10   */
11  import primerlabestrdatosconcesionariovehiculos.Clases.InventarioVehiculos;
12
13
14  public class ConcesionarioVehiculos {
15
16      public static void main(String[] args) {
17          InventarioVehiculos inventarioSucursal1 = new InventarioVehiculos();
18          inventarioSucursal1.agregarVehiculo("Bugatti");
19          inventarioSucursal1.agregarVehiculo("Ferrari");
20          inventarioSucursal1.agregarVehiculo("Porsche");
21          inventarioSucursal1.agregarVehiculo("Maserati");
22
23          InventarioVehiculos inventarioSucursal2 = new InventarioVehiculos();
24          inventarioSucursal2.agregarVehiculo("Alfa Romeo");
25          inventarioSucursal2.agregarVehiculo("Mercedes");
26          inventarioSucursal2.agregarVehiculo("BMW");
27          inventarioSucursal2.agregarVehiculo("Mazda");
28

```

Asignatura	Datos del alumno	Fecha
Estructura De Datos	Apellidos: De Mendoza	09/03/2024
	Nombre: Alejandro	

7. Ahora procedo a desarrollar las operaciones que voy a efectuar con el inventario de la sucursal 1 y para esto la primera operación se basa en llamar al método **contarVehiculos()** que imprime el total de vehículos en la primera sucursal y que imprime el numero 4 correspondiente a los vehículos Bugatti, Ferrari, Porsche y Maserati. A continuación, la imagen respectiva:



```

12
13
14 public class ConcesionarioVehiculos {
15
16     public static void main(String[] args) {
17         InventarioVehiculos inventarioSucursal1 = new InventarioVehiculos();
18         inventarioSucursal1.agregarVehiculo("Bugatti");
19         inventarioSucursal1.agregarVehiculo("Ferrari");
20         inventarioSucursal1.agregarVehiculo("Porsche");
21         inventarioSucursal1.agregarVehiculo("Maserati");
22
23         InventarioVehiculos inventarioSucursal2 = new InventarioVehiculos();
24         inventarioSucursal2.agregarVehiculo("Alfa Romeo");
25         inventarioSucursal2.agregarVehiculo("Mercedes");
26         inventarioSucursal2.agregarVehiculo("BMW");
27         inventarioSucursal2.agregarVehiculo("Mazda");
28
29         inventarioSucursal1.contarVehiculos();

```

8. Ahora procedo a llamar el método **inventarioSucursal1.mostrarVehiculoEnPosicion()** (en este caso se colocó el número 3, pero se puede elegir el que se desee), y en este caso muestra el vehículo en la posición 3 de la lista que es Maserati. Ahora si la numeración comienza en 1 imprime Ferrari y por otra parte si la numeración comienza en 0 imprime Bugatti. A continuación, la imagen respectiva:

Asignatura	Datos del alumno	Fecha
Estructura De Datos	Apellidos: De Mendoza	09/03/2024
	Nombre: Alejandro	

```

12
13
14 public class ConcesionarioVehiculos {
15
16     public static void main(String[] args) {
17         InventarioVehiculos inventarioSucursal1 = new InventarioVehiculos();
18         inventarioSucursal1.agregarVehiculo("Bugatti");
19         inventarioSucursal1.agregarVehiculo("Ferrari");
20         inventarioSucursal1.agregarVehiculo("Porsche");
21         inventarioSucursal1.agregarVehiculo("Maserati");
22
23         InventarioVehiculos inventarioSucursal2 = new InventarioVehiculos();
24         inventarioSucursal2.agregarVehiculo("Alfa Romeo");
25         inventarioSucursal2.agregarVehiculo("Mercedes");
26         inventarioSucursal2.agregarVehiculo("BMW");
27         inventarioSucursal2.agregarVehiculo("Mazda");
28
29         inventarioSucursal1.contarVehiculos();
30         inventarioSucursal1.mostrarVehiculoEnPosicion(3);

```

9. Se procede entonces a desarrollar el método `inventarioSucursal1.verificarVehiculo` (En este caso se tomó como nombre del vehículo "Porsche", pero se puede elegir el que se requiera) y lo que hace es busca si Porsche está en la lista, entonces si está, imprime "El vehículo Porsche está en el inventario." Y en caso contrario imprime "El vehículo Porsche no está en el inventario.". A continuación, la imagen respectiva:

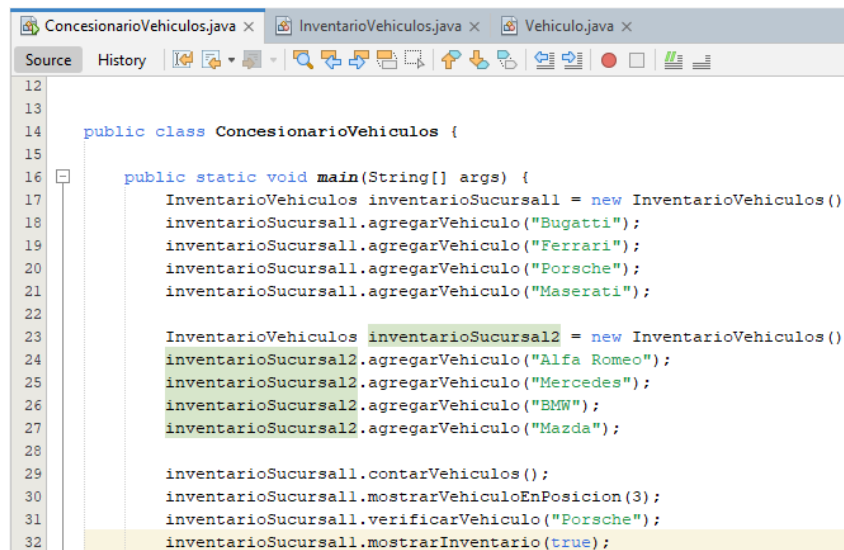
```

12
13
14 public class ConcesionarioVehiculos {
15
16     public static void main(String[] args) {
17         InventarioVehiculos inventarioSucursal1 = new InventarioVehiculos();
18         inventarioSucursal1.agregarVehiculo("Bugatti");
19         inventarioSucursal1.agregarVehiculo("Ferrari");
20         inventarioSucursal1.agregarVehiculo("Porsche");
21         inventarioSucursal1.agregarVehiculo("Maserati");
22
23         InventarioVehiculos inventarioSucursal2 = new InventarioVehiculos();
24         inventarioSucursal2.agregarVehiculo("Alfa Romeo");
25         inventarioSucursal2.agregarVehiculo("Mercedes");
26         inventarioSucursal2.agregarVehiculo("BMW");
27         inventarioSucursal2.agregarVehiculo("Mazda");
28
29         inventarioSucursal1.contarVehiculos();
30         inventarioSucursal1.mostrarVehiculoEnPosicion(3);
31         inventarioSucursal1.verificarVehiculo("Porsche");

```

Asignatura	Datos del alumno	Fecha
Estructura De Datos	Apellidos: De Mendoza	09/03/2024
	Nombre: Alejandro	

10. Ahora se procede a llamar al método **inventarioSucursal1.mostrarInventario(true)** el cual es el método que imprime la lista de vehículos de la primera sucursal en pantalla (Bugatti, Ferrari, Porsche y Maserati). A continuación, la imagen respectiva:



```

12
13
14 public class ConcesionarioVehiculos {
15
16     public static void main(String[] args) {
17         InventarioVehiculos inventarioSucursal1 = new InventarioVehiculos();
18         inventarioSucursal1.agregarVehiculo("Bugatti");
19         inventarioSucursal1.agregarVehiculo("Ferrari");
20         inventarioSucursal1.agregarVehiculo("Porsche");
21         inventarioSucursal1.agregarVehiculo("Maserati");
22
23         InventarioVehiculos inventarioSucursal2 = new InventarioVehiculos();
24         inventarioSucursal2.agregarVehiculo("Alfa Romeo");
25         inventarioSucursal2.agregarVehiculo("Mercedes");
26         inventarioSucursal2.agregarVehiculo("BMW");
27         inventarioSucursal2.agregarVehiculo("Mazda");
28
29         inventarioSucursal1.contarVehiculos();
30         inventarioSucursal1.mostrarVehiculoEnPosicion(3);
31         inventarioSucursal1.verificarVehiculo("Porsche");
32         inventarioSucursal1.mostrarInventario(true);

```

11. Se procede entonces a ejecutar las acciones de inserción y eliminación de vehículos entonces para el proceso de inserción se desarrolla el método de **inventarioSucursal1.insertarVehiculoEnPosicion**(En este caso se tomó como ejemplo el vehículo con el nombre de "Lamborghini" en la posición **número 2**) por ende como la numeración empieza en 0 en este caso la lista queda de la siguiente manera: Bugatti, Ferrari, Lamborghini, Porsche y Maserati. A continuación, la imagen respectiva:

Asignatura	Datos del alumno	Fecha
Estructura De Datos	Apellidos: De Mendoza	09/03/2024
	Nombre: Alejandro	

```

ConcesionarioVehiculos.java x InventarioVehiculos.java x Vehiculo.java x
Source History
12
13
14
15
16 public class ConcesionarioVehiculos {
17
18     public static void main(String[] args) {
19         InventarioVehiculos inventarioSucursal1 = new InventarioVehiculos();
20         inventarioSucursal1.agregarVehiculo("Bugatti");
21         inventarioSucursal1.agregarVehiculo("Ferrari");
22         inventarioSucursal1.agregarVehiculo("Porsche");
23         inventarioSucursal1.agregarVehiculo("Maserati");
24
25         InventarioVehiculos inventarioSucursal2 = new InventarioVehiculos();
26         inventarioSucursal2.agregarVehiculo("Alfa Romeo");
27         inventarioSucursal2.agregarVehiculo("Mercedes");
28         inventarioSucursal2.agregarVehiculo("BMW");
29         inventarioSucursal2.agregarVehiculo("Mazda");
30
31         inventarioSucursal1.contarVehiculos();
32         inventarioSucursal1.mostrarVehiculoEnPosicion(3);
33         inventarioSucursal1.verificarVehiculo("Porsche");
34         inventarioSucursal1.mostrarInventario(true);
35
36         inventarioSucursal1.insertarVehiculoEnPosicion("Lamborghini", 2);

```

12. Ahora procediendo con el método de eliminación se ejecuta el desarrollo del método **inventarioSucursal1.eliminarVehiculo**(En este caso se tomó como ejemplo el vehículo con el nombre de "Lamborghini") y lo que hace este método es buscar y eliminar Lamborghini de la lista. A continuación, la imagen respectiva:

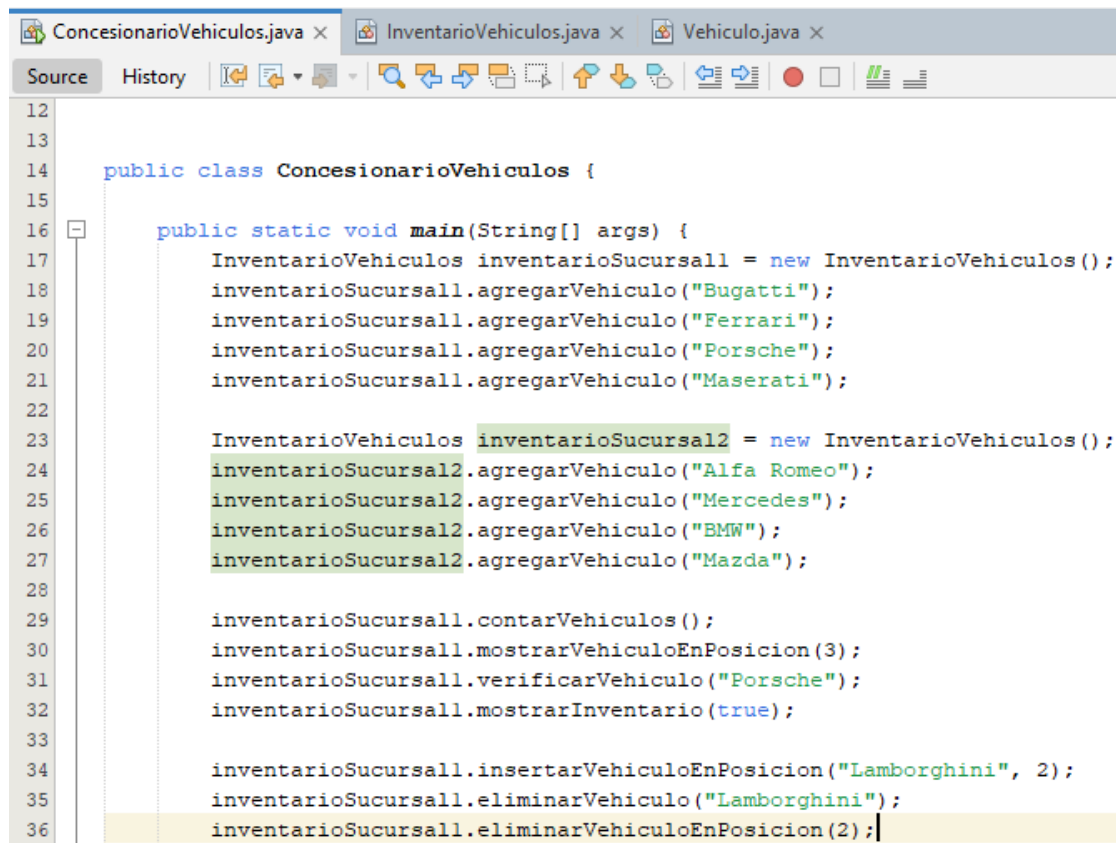
```

ConcesionarioVehiculos.java x InventarioVehiculos.java x Vehiculo.java x
Source History
12
13
14
15
16 public class ConcesionarioVehiculos {
17
18     public static void main(String[] args) {
19         InventarioVehiculos inventarioSucursal1 = new InventarioVehiculos();
20         inventarioSucursal1.agregarVehiculo("Bugatti");
21         inventarioSucursal1.agregarVehiculo("Ferrari");
22         inventarioSucursal1.agregarVehiculo("Porsche");
23         inventarioSucursal1.agregarVehiculo("Maserati");
24
25         InventarioVehiculos inventarioSucursal2 = new InventarioVehiculos();
26         inventarioSucursal2.agregarVehiculo("Alfa Romeo");
27         inventarioSucursal2.agregarVehiculo("Mercedes");
28         inventarioSucursal2.agregarVehiculo("BMW");
29         inventarioSucursal2.agregarVehiculo("Mazda");
30
31         inventarioSucursal1.contarVehiculos();
32         inventarioSucursal1.mostrarVehiculoEnPosicion(3);
33         inventarioSucursal1.verificarVehiculo("Porsche");
34         inventarioSucursal1.mostrarInventario(true);
35
36         inventarioSucursal1.insertarVehiculoEnPosicion("Lamborghini", 2);
37         inventarioSucursal1.eliminarVehiculo("Lamborghini");

```

Asignatura	Datos del alumno	Fecha
Estructura De Datos	Apellidos: De Mendoza	09/03/2024
	Nombre: Alejandro	

13. Se procede entonces a desarrollar el método **inventarioSucursal1.eliminarVehiculoEnPosicion**(En este caso se tomó como ejemplo la posición número 2, pero de nuevo se puede elegir la que se considere), y lo que hace el método es elimina el vehículo en la posición 2 según lo que especifico. A continuación, la imagen respectiva:



```

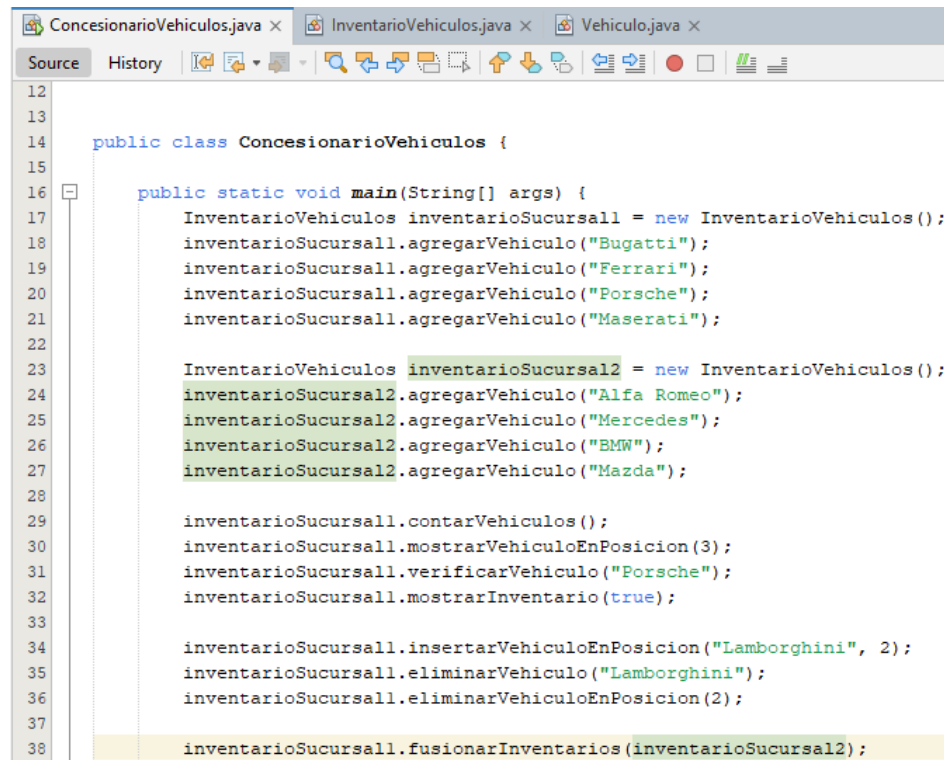
12
13
14 public class ConcesionarioVehiculos {
15
16     public static void main(String[] args) {
17         InventarioVehiculos inventarioSucursal1 = new InventarioVehiculos();
18         inventarioSucursal1.agregarVehiculo("Bugatti");
19         inventarioSucursal1.agregarVehiculo("Ferrari");
20         inventarioSucursal1.agregarVehiculo("Porsche");
21         inventarioSucursal1.agregarVehiculo("Maserati");
22
23         InventarioVehiculos inventarioSucursal2 = new InventarioVehiculos();
24         inventarioSucursal2.agregarVehiculo("Alfa Romeo");
25         inventarioSucursal2.agregarVehiculo("Mercedes");
26         inventarioSucursal2.agregarVehiculo("BMW");
27         inventarioSucursal2.agregarVehiculo("Mazda");
28
29         inventarioSucursal1.contarVehiculos();
30         inventarioSucursal1.mostrarVehiculoEnPosicion(3);
31         inventarioSucursal1.verificarVehiculo("Porsche");
32         inventarioSucursal1.mostrarInventario(true);
33
34         inventarioSucursal1.insertarVehiculoEnPosicion("Lamborghini", 2);
35         inventarioSucursal1.eliminarVehiculo("Lamborghini");
36         inventarioSucursal1.eliminarVehiculoEnPosicion(2);

```

14. Ahora para ejecutar el proceso de concatenación desarrollé el método **inventarioSucursal1.fusionarInventarios(inventarioSucursal2)**, que genera una unión entre los vehículos de inventarioSucursal2 a inventarioSucursal1 y en este caso la respuesta son todos los vehículos de

Asignatura	Datos del alumno	Fecha
Estructura De Datos	Apellidos: De Mendoza	09/03/2024
	Nombre: Alejandro	

inventarioSucursal1 más los de inventarioSucursal2. A continuación, la imagen respectiva:



```

12
13
14 public class ConcesionarioVehiculos {
15
16     public static void main(String[] args) {
17         InventarioVehiculos inventarioSucursal1 = new InventarioVehiculos();
18         inventarioSucursal1.agregarVehiculo("Bugatti");
19         inventarioSucursal1.agregarVehiculo("Ferrari");
20         inventarioSucursal1.agregarVehiculo("Porsche");
21         inventarioSucursal1.agregarVehiculo("Maserati");
22
23         InventarioVehiculos inventarioSucursal2 = new InventarioVehiculos();
24         inventarioSucursal2.agregarVehiculo("Alfa Romeo");
25         inventarioSucursal2.agregarVehiculo("Mercedes");
26         inventarioSucursal2.agregarVehiculo("BMW");
27         inventarioSucursal2.agregarVehiculo("Mazda");
28
29         inventarioSucursal1.contarVehiculos();
30         inventarioSucursal1.mostrarVehiculoEnPosicion(3);
31         inventarioSucursal1.verificarVehiculo("Porsche");
32         inventarioSucursal1.mostrarInventario(true);
33
34         inventarioSucursal1.insertarVehiculoEnPosicion("Lamborghini", 2);
35         inventarioSucursal1.eliminarVehiculo("Lamborghini");
36         inventarioSucursal1.eliminarVehiculoEnPosicion(2);
37
38         inventarioSucursal1.fusionarInventarios(inventarioSucursal2);

```

Es importante indicar que junto a este método se creó el método de igual manera **inventarioSucursal1.mostrarInventario(false)**, que lo ejecuta es la muestra del inventario combinado después de la fusión. A continuación, la imagen respectiva:

Asignatura	Datos del alumno	Fecha
Estructura De Datos	Apellidos: De Mendoza	09/03/2024
	Nombre: Alejandro	

```

12
13
14 public class ConcesionarioVehiculos {
15
16     public static void main(String[] args) {
17         InventarioVehiculos inventarioSucursal1 = new InventarioVehiculos();
18         inventarioSucursal1.agregarVehiculo("Bugatti");
19         inventarioSucursal1.agregarVehiculo("Ferrari");
20         inventarioSucursal1.agregarVehiculo("Porsche");
21         inventarioSucursal1.agregarVehiculo("Maserati");
22
23         InventarioVehiculos inventarioSucursal2 = new InventarioVehiculos();
24         inventarioSucursal2.agregarVehiculo("Alfa Romeo");
25         inventarioSucursal2.agregarVehiculo("Mercedes");
26         inventarioSucursal2.agregarVehiculo("BMW");
27         inventarioSucursal2.agregarVehiculo("Mazda");
28
29         inventarioSucursal1.contarVehiculos();
30         inventarioSucursal1.mostrarVehiculoEnPosicion(3);
31         inventarioSucursal1.verificarVehiculo("Porsche");
32         inventarioSucursal1.mostrarInventario(true);
33
34         inventarioSucursal1.insertarVehiculoEnPosicion("Lamborghini", 2);
35         inventarioSucursal1.eliminarVehiculo("Lamborghini");
36         inventarioSucursal1.eliminarVehiculoEnPosicion(2);
37
38         inventarioSucursal1.fusionarInventarios(inventarioSucursal2);
39         inventarioSucursal1.mostrarInventario(false);

```

15. Por último, se crea el método `inventarioSucursal1.actualizarVehiculo("Bugatti", "Tesla")`, que como su nombre lo indica la acción que ejecuta es la de actualizar un vehículo de la lista por otro que se considere en este caso se tomó como ejemplo el reemplazo de Bugatti por Tesla. A continuación, la imagen respectiva:

```

12
13
14 public class ConcesionarioVehiculos {
15
16     public static void main(String[] args) {
17         InventarioVehiculos inventarioSucursal1 = new InventarioVehiculos();
18         inventarioSucursal1.agregarVehiculo("Bugatti");
19         inventarioSucursal1.agregarVehiculo("Ferrari");
20         inventarioSucursal1.agregarVehiculo("Porsche");
21         inventarioSucursal1.agregarVehiculo("Maserati");
22
23         InventarioVehiculos inventarioSucursal2 = new InventarioVehiculos();
24         inventarioSucursal2.agregarVehiculo("Alfa Romeo");
25         inventarioSucursal2.agregarVehiculo("Mercedes");
26         inventarioSucursal2.agregarVehiculo("BMW");
27         inventarioSucursal2.agregarVehiculo("Mazda");
28
29         inventarioSucursal1.contarVehiculos();
30         inventarioSucursal1.mostrarVehiculoEnPosicion(3);
31         inventarioSucursal1.verificarVehiculo("Porsche");
32         inventarioSucursal1.mostrarInventario(true);
33
34         inventarioSucursal1.insertarVehiculoEnPosicion("Lamborghini", 2);
35         inventarioSucursal1.eliminarVehiculo("Lamborghini");
36         inventarioSucursal1.eliminarVehiculoEnPosicion(2);
37
38         inventarioSucursal1.fusionarInventarios(inventarioSucursal2);
39         inventarioSucursal1.mostrarInventario(false);
40         inventarioSucursal1.actualizarVehiculo("Bugatti", "Tesla");
41     }
42 }

```

Asignatura	Datos del alumno	Fecha
Estructura De Datos	Apellidos: De Mendoza	09/03/2024
	Nombre: Alejandro	

16. Y con esto se da por finalizado el desarrollo y explicación de esta clase, ahora procedo a dar a conocer los resultados obtenidos en la consola con base en el desarrollo implementado.

Resultados Obtenidos En La Consola De NETBEANS

A continuación, detallo la lista de resultados obtenidos en la consola de la plataforma NetBeans sobre todo el desarrollo del laboratorio con el fin de dar a conocer de manera clara todos los puntos que fueron resueltos en su totalidad frente al laboratorio expuesto y aplicados a un escenario real. A continuación, los puntos a desarrollar junto con la imagen de los resultados obtenidos frente cada punto:

Puntos por desarrollar:

- Contar el número de elementos que tiene la lista.
- Mostrar el elemento que hay en una posición concreta de la lista.
- Comprobar si un elemento está en la lista.
- Imprimir los elementos que contiene la lista.
- Insertar un elemento nuevo.
- Sacar un elemento concreto de la lista.
- Sacar el elemento que ocupa una posición en la lista.
- Concatenar dos listas.
- Reemplazar un elemento de la lista.

Asignatura	Datos del alumno	Fecha
Estructura De Datos	Apellidos: De Mendoza	09/03/2024
	Nombre: Alejandro	

Imagen de desarrollo de los puntos en consola NetBeans:

```

Output - PrimerLabEstrDatosConcesionarioVehiculos (run)

El numero total de vehiculos en la lista de el inventario: 4.

El vehiculo en la posicion 3 es: Maserati.

El vehiculo 'Porsche' esta en el inventario.

Modelos en la lista del inventario:
- Bugatti
- Ferrari
- Porsche
- Maserati

Agregando vehiculo nuevo 'Lamborghini' en la posicion 2.
- Bugatti
- Ferrari
- Lamborghini
- Porsche
- Maserati

Se ha eliminado el vehiculo 'Lamborghini' como elemento concreto de la lista de vehiculos.
- Bugatti
- Ferrari
- Porsche
- Maserati

Se elimino el vehiculo en la posicion 2.
- Bugatti
- Ferrari
- Maserati

Inventarios fusionados o listas de inventarios concatenadas:
- Bugatti
- Ferrari
- Maserati
- Alfa Romeo
- Mercedes
- BMW
- Mazda

Se actualizo 'Bugatti' por 'Tesla'.
- Tesla
- Ferrari
- Maserati
- Alfa Romeo
- Mercedes
- BMW
- Mazda

BUILD SUCCESSFUL (total time: 0 seconds)

```

Y con esto se da por finalizado este desarrollo del primer laboratorio de Estructura de Datos en su totalidad. Muchas gracias por su valiosa atención y lectura.

CONCLUSIÓN

Asignatura	Datos del alumno	Fecha
Estructura De Datos	Apellidos: De Mendoza	09/03/2024
	Nombre: Alejandro	

Desde mi punto de vista el desarrollo de este laboratorio representó un ejercicio integral para mí en la aplicación de listas doblemente enlazadas en Java, las cuales las aplique a un entorno real que en este caso fue en la gestión de inventarios de vehículos dentro de un concesionario como lo explique en detalle en su anterioridad y es por esto por lo que, a lo largo de la implementación, desarrolle conceptos avanzados de estructuras de datos dinámicas, permitiendo manipular el inventario de manera eficiente y optimizada en este caso de un concesionario de vehículos de alta gama. Algunas de las ventajas de las que me pude percatar en el uso de este desarrollo fueron:

- Acceso bidireccional: Se puede recorrer la lista tanto hacia adelante como hacia atrás, permitiendo una mayor flexibilidad en la manipulación del inventario (explicado de igual manera en el aula virtual).
- Inserciones y eliminaciones eficientes: Al contar con punteros hacia el nodo anterior y siguiente, las operaciones de agregar o eliminar vehículos en posiciones específicas se realizan en tiempo constante en muchos casos.
- Optimización de búsquedas y actualizaciones: Se implementaron métodos que permiten localizar y modificar vehículos sin necesidad de recorrer toda la estructura innecesariamente.
- Facilidad en la fusión de inventarios: Gracias a la naturaleza dinámica de la lista doblemente enlazada, pude unir inventarios de distintas sucursales sin la necesidad de copiar y reestructurar grandes bloques de memoria.

Asignatura	Datos del alumno	Fecha
Estructura De Datos	Apellidos: De Mendoza	09/03/2024
	Nombre: Alejandro	

Debo adicionalmente indicar que el sistema desarrollado permitió realizar diversas operaciones sobre la estructura de la lista doblemente enlazada, asegurando una gestión eficiente del inventario, basándonos en las siguientes funcionalidades:

- Agregar vehículos: Se insertan al final de la lista o en una posición específica sin necesidad de reorganizar elementos.
- Eliminar vehículos: Se puede eliminar un vehículo por nombre o por su posición dentro de la lista, reajustando los punteros entre nodos.
- Actualizar vehículos: Se recorre la lista hasta encontrar el vehículo a modificar y se reemplaza su valor.
- Buscar vehículos: Se implementó una búsqueda secuencial optimizada que aprovecha la doble dirección de los nodos.
- Mostrar inventario: Me permite visualizar el inventario en orden normal o invertido, demostrando la ventaja del doble enlace.
- Fusionar inventarios: Se conectan listas de diferentes sucursales sin necesidad de duplicar elementos o utilizar estructuras auxiliares.

Ahora desde un punto de vista técnico y en términos prácticos, este proyecto permitió reforzar conceptos fundamentales de estructuras de datos, manipulación de nodos y eficiencia en la administración de información, por lo que el sistema desarrollado podría aplicarse en múltiples escenarios reales como:

- Gestión de inventarios en concesionarios o tiendas

Asignatura	Datos del alumno	Fecha
Estructura De Datos	Apellidos: De Mendoza	09/03/2024
	Nombre: Alejandro	

- Administración de pedidos en sistemas de logística
- Historial de navegación en aplicaciones web
- Sistemas de edición (como editores de texto o imágenes) donde se requiere retroceder y avanzar entre elementos

Para finalizar quiero indicar que este desarrollo para mi representó una experiencia completa en la implementación y manipulación de listas doblemente enlazadas en Java, permitiendo comprender su importancia en la gestión eficiente de datos dinámicos. De mi parte se considera se logró un sistema escalable y funcional que puede ser extendido en el futuro con integración a bases de datos, interfaces gráficas o incluso algoritmos avanzados de búsqueda y ordenación. En conclusión, este proyecto permitió consolidar conocimientos esenciales en estructuras de datos avanzadas, programación orientada a objetos y optimización de algoritmos, dejando una base sólida para futuras mejoras en el ámbito de la administración de información y desarrollo de software. lo que es fundamental para proyectos más grandes y complejos en mi futuro como ingeniero informático, muchas gracias, respetado profesor.

BIBLIOGRAFÍA

A continuación, la bibliografía implementada:

- ✓ Tema 1. Introducción a la programación en Java. Tema 2. Tipos abstractos de datos. Tema 3. Estructuras de datos lineales. Tema 4. ED lineales: pilas y colas.

Asignatura	Datos del alumno	Fecha
Estructura De Datos	Apellidos: De Mendoza	09/03/2024
	Nombre: Alejandro	

- ✓ Clases virtuales con el profesor Ing. Edwin Eduardo Millán Rojas.
- ✓ Material de estudio del aula.